

DATA 298A • Master's Project I

Project List — Section 21

Each team must choose a different topic (first-come, first-served).

Teams of 4–5 • two semesters • end-to-end system • minimum four models built, benchmarked and compared.

What every project on this list must deliver

A two-semester project for 4–5 graduate students must deliver a working system, not a notebook.

End-to-end system. Data, model, serving or control, and interface layers. Another person must be able to run it.

Four models, all built by you. Build four genuinely different models — not hyperparameter variants. One in 298A; three in 298B.

Plus a reproduced baseline. Reproduce one published benchmark result.

Benchmarking and evaluation. On a named benchmark, report metrics, multiple seeds, error bars, an ablation, wins and failures.

Reproducibility. Pin code and environment; log seeds and runs; document the hardware budget.

Failure analysis. Classify failures and conditions; this is weighted like accuracy.

Each topic lists at least four models and an evaluation plan; stronger substitutions need approval.

YC topics use Y Combinator's Fall 2026 RFS: unsolved, fundable problems.

1. Continual Learning for Embodied Agents

Design an embodied agent that acquires a sequence of physical skills without catastrophic forgetting, then quantify the stability–plasticity trade-off across methods.

Why it matters: Current embodied systems retrain from scratch for every new skill. A long-lived agent must accumulate skills the way a technician does.

Technologies: NVIDIA Isaac Sim / MuJoCo / PyBullet, RL (PPO, SAC), continual-learning methods (EWC, LwF, experience replay, progressive nets, LoRA adapters), PyTorch.

Tasks:

- Build a simulated manipulator/locomotion agent and a curriculum of 6+ distinct tasks
- Train sequentially and measure forgetting on all previously learned tasks
- Implement and compare four continual-learning strategies under identical budgets
- Ablate memory size, task order, and rehearsal ratio
- Package as a reproducible benchmark harness with logged trajectories

Models to build and compare (5 minimum):

- Naive sequential fine-tuning (lower bound)
- Elastic Weight Consolidation or Synaptic Intelligence
- Experience-replay / generative-rehearsal policy
- Parameter-isolation policy (progressive nets or adapter routing)
- Multi-task joint training (upper bound)

Evaluation and benchmarking: Average success rate, backward transfer (forgetting), forward transfer, wall-clock and memory cost; compare against published CORE50 / Continual-World numbers.

Outcome: A continual-learning embodied agent plus a public benchmark report showing which method actually retains skills at what cost.

2. Vision-Language-Action Models for Long-Horizon Manipulation

Build an embodied system that turns free-form natural-language instructions into multi-step physical action, and measure where language grounding breaks down.

Why it matters: Instruction-following robots fail on compositional, long-horizon commands. The interesting research is in the failure taxonomy, not the demo.

Technologies: VLMs (SigLIP, CLIP, Qwen-VL), open VLA checkpoints (OpenVLA, Octo), diffusion policies, Isaac Sim / MuJoCo, imitation and offline RL, PyTorch.

Tasks:

- Assemble a language-annotated demonstration dataset in simulation
- Implement a perception-to-action pipeline with an explicit language grounding stage
- Train four policy families and evaluate on held-out instruction phrasings
- Stress-test on unseen objects, distractors, and compositional instructions
- Build an error taxonomy that attributes failures to perception, grounding, or control

Models to build and compare (4 minimum):

- End-to-end VLA policy (fine-tuned open checkpoint)
- Modular LLM planner + learned low-level skill library
- Behaviour-cloning baseline without language conditioning
- Diffusion policy with language conditioning

Evaluation and benchmarking: Task success, subgoal completion rate, instruction-generalization gap, sample efficiency; compare with reported OpenVLA / Octo results.

Outcome: A working VLA system and a defensible account of which architecture generalizes to new language and why.

3. Fleet Learning: Continual Improvement from Deployed Robot Data

Build the loop that turns data from many deployed robots into a better shared policy — deciding which experience is worth keeping and how to avoid making every robot worse at once.

Why it matters: A single robot learns slowly; a fleet learns fast, but only if data selection, aggregation and rollout gating are right. Getting it wrong degrades the whole fleet simultaneously.

Technologies: Isaac Sim multi-robot simulation, offline and off-policy RL, prioritized data selection, federated or centralized aggregation, policy versioning, canary rollout, drift detection.

Tasks:

- Simulate a fleet of robots operating across varied environments
- Build the data loop: what is logged, what is kept, what is discarded
- Aggregate into a shared policy and gate its rollout on evaluation
- Compare four data-selection and aggregation strategies
- Test the failure case: one robot's bad data degrading the fleet

Models to build and compare (4 minimum):

- Single-robot policy, no fleet learning (baseline)
- Naive pooled retraining over all logged data
- Prioritized data-selection policy
- Federated aggregation with per-robot personalization

Evaluation and benchmarking: Fleet-average success rate across rollout generations, sample efficiency versus single-robot learning, regression rate after a rollout, robustness to anomalous or poisoned robot data.

Outcome: A fleet learning loop with a rollout gate that demonstrably prevents fleet-wide regression.

4. Multiplayer Agents: Shared Live Agent Sessions [YC]

Build the collaboration layer that agent tooling currently lacks: multiple humans and multiple agents working in one live, shared session with concurrent edits, hand-off and redirection.

Why it matters: Y Combinator names this explicitly in its Fall 2026 Requests for Startups: AI is still single-player while agent tasks now run for hours or days across whole teams.

Technologies: CRDTs / operational transforms, WebSockets, LLM agent frameworks (LangGraph, CrewAI), vector and structured memory, distributed state, conflict-resolution policy learning.

Tasks:

- Design a shared session model with per-participant permissions and live state
- Implement conflict detection and resolution when two actors redirect the same agent
- Build hand-off, interruption and rollback primitives with a full audit trail
- Train/evaluate models that summarize session state for a joining participant
- Run a multi-user study measuring task completion versus single-player baseline

Models to build and compare (4 minimum):

- Conflict-resolution / merge-arbitration model
- Session-state summarization and catch-up model
- Intent-attribution model (which participant does an instruction serve)
- Task-decomposition and routing model across humans and agents

Evaluation and benchmarking: Task completion time, merge-conflict rate, instruction-attribution accuracy, human preference scores against a single-player agent baseline.

Outcome: A multiplayer agent platform with measured evidence that shared sessions beat isolated ones.

5. Autonomous Physical-World Data Acquisition and Active Sensing [YC]

Build an autonomous sensing system that decides where, when and how to collect physical-world data, then updates a predictive world model under energy, bandwidth and safety constraints.

Why it matters: YC Fall 2026 says sparse real-world data is the bottleneck in energy, agriculture, logistics and construction. Passive collection wastes measurements where uncertainty is already low.

Technologies: Active learning, Bayesian experimental design, spatiotemporal graph models, mobile sensors in simulation, uncertainty calibration, constrained planning and public environmental data.

Tasks:

- Build a multi-source spatiotemporal benchmark with controllable sensing gaps
- Train an uncertainty-aware world model that identifies information deficits
- Plan adaptive sensor queries under energy and bandwidth budgets
- Compare four acquisition policies with identical sample and compute budgets
- Stress-test distribution shift, sensor failure and delayed observations

Models to build and compare (4 minimum):

- Gaussian-process or neural-process acquisition model
- Spatiotemporal graph world model with calibrated uncertainty
- Reinforcement-learning active-sensing policy
- Information-gain or submodular-optimization baseline

Evaluation and benchmarking: Forecast RMSE and CRPS versus samples acquired, information gain per joule, rare-event coverage, calibration, robustness to sensor loss and latency.

Outcome: An active-sensing platform that learns a better physical-world model with fewer measurements and a reproducible cost-quality frontier.

6. Multi-Agent Debugging and Coordination-Failure Detection

Multi-agent systems fail silently through loops, dropped hand-offs and mutual misunderstanding. Build the observability and automated diagnosis layer for them.

Why it matters: Reliability, not capability, is what blocks agent deployment. There is no equivalent of a debugger for agent swarms.

Technologies: CrewAI / LangGraph / AutoGen-style workflows, distributed tracing (OpenTelemetry), trace-graph learning, anomaly detection, LLM-based root-cause reasoning.

Tasks:

- Instrument a multi-agent framework to emit structured execution traces
- Construct a labelled corpus of injected coordination failures
- Train four detectors to classify failure mode from a trace
- Generate human-readable root-cause explanations and repair suggestions
- Close the loop: apply repairs and measure recovered task success

Models to build and compare (4 minimum):

- Sequence model over trace events (transformer or state-space)
- Graph neural network over the agent interaction graph
- LLM root-cause reasoner with retrieval over past incidents
- Statistical anomaly detector as a baseline

Evaluation and benchmarking: Failure-detection F1 by mode, time-to-detect, root-cause accuracy against injected ground truth, task success recovered after automated repair.

Outcome: An agent debugger that detects, explains and repairs coordination failures with measured accuracy.

7. AI Co-Scientist for Automated ML Experimentation

Build an agent that reads a research goal, proposes hypotheses, designs and runs experiments, analyses results and decides what to try next — with humans auditing every step.

Why it matters: Research iteration is bottlenecked by human bandwidth on routine experimentation, not by ideas.

Technologies: LLM agents with tool use, AutoML and hyperparameter optimization (BOHB, ASHA), experiment tracking (MLflow, Weights & Biases), sandboxed execution, statistical testing.

Tasks:

- Define an experiment specification language and a safe execution sandbox
- Implement hypothesis generation, experiment design and result interpretation
- Add statistical rigor: power analysis, multiple-comparison correction, seed variance
- Benchmark four search/planning strategies on identical compute budgets
- Validate by reproducing a known published result end to end

Models to build and compare (4 minimum):

- LLM planner-agent with tool use
- Bayesian optimization search strategy
- Bandit / successive-halving scheduler
- Learned surrogate performance predictor

Evaluation and benchmarking: Best result found per GPU-hour, reproduction fidelity on a known paper, false-discovery rate, human expert rating of generated analyses.

Outcome: A co-scientist that measurably beats a competent graduate student per unit of compute on a defined search space.

8. Adversarial Reinforcement Learning for Cloud Penetration Testing

Train RL agents that autonomously discover misconfigurations and privilege-escalation paths in a simulated cloud estate, with a defender agent trained against them.

Why it matters: Manual pen-testing does not scale to cloud estates that change hourly. Self-play produces attack paths humans do not think of.

Technologies: Gym-style cloud environment, Stable-Baselines3, self-play and population-based training, attack-graph modelling, IAM/policy simulation, MITRE ATT&CK mapping.

Tasks:

- Build a high-fidelity simulated cloud environment with IAM, network and service layers
- Model the attack surface as a graph with realistic reward shaping
- Train attacker agents and a co-evolving defender via self-play
- Compare four agent designs on exploit discovery under identical budgets
- Map discovered paths to MITRE ATT&CK and validate against known CVE scenarios

Models to build and compare (4 minimum):

- Model-free deep RL attacker (PPO/DQN)
- Graph-neural-network policy over the attack graph
- LLM-guided planner with tool execution
- Classical planner / heuristic search baseline

Evaluation and benchmarking: Exploit discovery rate, steps-to-compromise, novel-path count versus a scripted scanner, defender mean-time-to-contain.

Outcome: An autonomous red-team system with a reproducible cloud benchmark and a defender trained against it.

10. Distributed Systems for Intermittent Offshore Compute Networks [YC]

Build a distributed control plane for offshore compute vessels that keeps jobs and data available despite intermittent links, vessel motion, power variation and regional failures.

Why it matters: YC Fall 2026 proposes compute flotillas as a global cloud. Their network behaves unlike a data center: partitions, variable power and migration costs are first-order constraints.

Technologies: Distributed consensus under partitions, erasure coding, delay-tolerant networking, workload replication, graph forecasting, failure injection and robust optimization.

Tasks:

- Build a flotilla network and power simulator with correlated disruptions
- Implement geo-replicated storage and checkpoint transfer over unstable links
- Predict link quality, power availability and vessel failure risk
- Schedule replication and migration under durability, latency and energy SLOs
- Inject compound partitions and compare recovery across four control policies

Models to build and compare (4 minimum):

- Probabilistic link and power availability forecaster
- Graph-neural failure-propagation model
- Risk-sensitive replication and migration policy
- Erasure-coded quorum or delay-tolerant heuristic baseline

Evaluation and benchmarking: Data durability, job completion, recovery point/time, WAN bytes, energy overhead and tail latency across historical storm-like traces.

Outcome: A partition-tolerant flotilla control plane with quantified durability and cost under realistic intermittent connectivity.

11. Non-Transformer LLM Architectures at Scale

Train and evaluate sub-quadratic alternatives to attention, and establish where they genuinely win and where they quietly lose.

Why it matters: Long-context inference cost is the binding constraint on deployed LLMs. The literature reports wins on throughput and is much quieter about recall.

Technologies: State-space models (Mamba, S4/S5), linear and sliding-window attention, RWKV, hybrid stacks, FlashAttention baselines, PyTorch, mixed-precision training.

Tasks:

- Implement four architectures under an identical training budget and tokenizer
- Pre-train at a comparable parameter scale on a shared corpus
- Evaluate on language modelling, long-context retrieval and reasoning
- Profile memory, latency and throughput versus sequence length
- Characterize the recall degradation that sub-quadratic models exhibit

Models to build and compare (4 minimum):

- Transformer baseline with FlashAttention
- Selective state-space model (Mamba-style)
- Linear or sliding-window attention variant
- Hybrid attention + SSM stack

Evaluation and benchmarking: Perplexity, needle-in-a-haystack and RULER long-context scores, throughput (tokens/sec) and peak memory versus context length, scaling-curve comparison.

Outcome: A rigorous architecture comparison with reproducible training code and an honest account of the trade-off.

12. Test-Time Compute Scaling and Reasoning-Budget Allocation

Study and productionize the newest lever in model quality: spending more compute at inference. Build a router that decides how much reasoning each query deserves.

Why it matters: Inference-time compute now buys more accuracy per dollar than retraining for many tasks, and nobody spends it adaptively.

Technologies: Chain-of-thought and self-consistency, best-of-N with reward models, tree/graph search over reasoning, process reward models, verifier models, serving infrastructure.

Tasks:

- Implement four inference-time scaling strategies over the same base model
- Train a process reward model or verifier to score partial reasoning
- Build a difficulty predictor that allocates budget per query
- Measure accuracy-versus-compute curves on math, code and reasoning benchmarks
- Deploy as a serving layer with latency SLOs and cost accounting

Models to build and compare (4 minimum):

- Self-consistency majority voting
- Best-of-N with a trained reward model
- Tree-search / lookahead reasoning
- Query-difficulty router that sets the budget adaptively

Evaluation and benchmarking: Accuracy versus FLOPs curves on GSM8K/MATH, HumanEval and GPQA; cost per solved problem; latency distribution under an SLO.

Outcome: An adaptive reasoning-budget router with measured Pareto improvement over fixed-budget inference.

13. Memory-Centric AI: Beyond Vector Databases

Design a memory architecture for long-lived agents with episodic, semantic and working memory, consolidation and forgetting — then show it beats flat vector retrieval.

Why it matters: Vector search is a lookup table, not a memory. Agents running for weeks need consolidation, revision and principled forgetting.

Technologies: LLMs, memory-augmented networks, structured and graph memory, hierarchical summarization, retrieval beyond nearest-neighbour, consolidation scheduling, conflict resolution.

Tasks:

- Design a typed memory schema with explicit write, consolidate and forget operations
- Implement conflict detection when new information contradicts stored memory
- Build four retrieval/memory strategies and evaluate on long-horizon dialogue and task benchmarks
- Measure memory growth, retrieval latency and staleness over simulated months of use
- Ablate consolidation frequency and forgetting policy

Models to build and compare (4 minimum):

- Flat vector RAG baseline
- Hierarchical summarization memory
- Knowledge-graph structured memory with entity resolution
- Learned memory controller (what to write, keep, forget)

Evaluation and benchmarking: LongMemEval / LoCoMo accuracy, contradiction rate, retrieval latency and storage growth over time, task success after simulated long deployment.

Outcome: A memory system with evidence that structure and consolidation outperform flat retrieval at long horizons.

14. Digital Twin for Autonomous Data-Center Operations

Build a physics-grounded digital twin of a data center that forecasts thermal and mechanical failures and closes the loop by controlling cooling and workload placement.

Why it matters: Consolidates two overlapping topics. Reactive maintenance and static cooling set-points waste power and cause outages.

Technologies: NVIDIA Omniverse, Cadence Reality DC (SJSU licence available), physics-based simulation, time-series forecasting, RL control, synthetic telemetry generation, PyTorch.

Tasks:

- Build the twin geometry, thermal model and synthetic telemetry pipeline
- Train failure-prediction models on degradation signatures
- Train a control policy for cooling and workload placement
- Compare four models on prediction horizon and control efficiency
- Validate on injected fault scenarios and report energy saved

Models to build and compare (4 minimum):

- Time-series anomaly / degradation detector
- Remaining-useful-life regression model
- Physics-informed neural surrogate of the thermal model
- RL control policy for cooling and placement

Evaluation and benchmarking: Failure-prediction lead time and precision/recall, PUE and energy delta versus a static baseline, thermal SLA violations, simulation-to-twin fidelity error.

Outcome: A closed-loop data-center twin with quantified energy savings and failure lead time.

15. GPU Cluster Failure Prediction and Self-Healing Training Jobs

Large training runs die from hardware faults, stragglers and silent data corruption. Build a system that predicts these events and reconfigures the job before it crashes.

Why it matters: At cluster scale, mean time between failures is measured in hours, and a naive restart discards hours of compute.

Technologies: Cluster telemetry (DCGM, NCCL logs, ECC counters), time-series and sequence models, checkpoint/restart and elastic training, straggler detection, fault injection.

Tasks:

- Build a cluster simulator with realistic fault and straggler injection
- Collect or synthesize telemetry with labelled failure events
- Train four predictors for imminent node, network and silent-corruption failures
- Implement automated mitigation: pre-emptive checkpoint, node drain, elastic reshard
- Measure goodput recovered versus a naive checkpoint-and-restart baseline

Models to build and compare (4 minimum):

- Time-series anomaly detector over hardware counters
- Sequence model over log events
- Gradient/loss-anomaly detector for silent data corruption
- Straggler-classification and rebalancing model

Evaluation and benchmarking: Failure precision/recall and lead time, goodput (useful GPU-hours / total), wasted-compute reduction, false-drain cost.

Outcome: A self-healing training controller with a measured goodput improvement under injected faults.

16. The Primer: An Adaptive Literacy and Numeracy Tutor [YC]

Build a tutor for young children that teaches reading, writing and arithmetic at the quality of a devoted private tutor, adapting continuously to the individual learner.

Why it matters: Named directly in the YC Fall 2026 RFS. One-to-one tutoring produces the largest effect sizes in education research and has never been affordable at scale.

Technologies: LLMs with pedagogical constraints, knowledge tracing (BKT, DKT, SAKT), speech recognition for read-aloud assessment, curriculum RL, content safety filtering, learning analytics.

Tasks:

- Build a skill graph and an item bank aligned to a real curriculum standard
- Implement knowledge tracing to estimate mastery per skill in real time
- Add a speech pipeline that scores oral reading fluency
- Compare four tutoring policies for what to teach next
- Run a controlled study with simulated and, if approved, real learners

Models to build and compare (4 minimum):

- Deep knowledge-tracing model of learner mastery
- Content-generation model constrained to reading level and skill
- Speech-recognition and fluency-scoring model for read-aloud
- Curriculum-sequencing policy (RL or bandit) selecting the next item

Evaluation and benchmarking: Learning gain versus a fixed-sequence baseline, mastery-prediction AUC, oral-fluency scoring agreement with human raters, age-appropriateness and safety audit.

Outcome: An adaptive tutor with measured learning gains and an auditable safety record for child users.

17. Privacy-Preserving Personal AI Operating System [YC]

Build a multimodal personal agent that learns routines and preferences, plans across calendar, messages and local files, and executes bounded actions without exporting raw personal data.

Why it matters: YC Fall 2026 asks for AI-powered consumer products at massive scale. Trustworthy personalization, continual operation and low per-user cost remain open systems problems.

Technologies: On-device models, retrieval and long-term memory, preference learning, tool-using agents, policy engines, privacy-preserving analytics, synthetic user simulators.

Tasks:

- Build a consented personal-data sandbox and long-horizon task benchmark
- Implement memory with deletion, provenance and conflict resolution
- Learn preferences from explicit feedback and observed behavior
- Add a planner-executor with per-tool permissions and rollback
- Compare quality, privacy, latency and cost across local and cloud variants

Models to build and compare (4 minimum):

- Retrieval-augmented hierarchical planner
- Recurrent or state-space routine and preference model
- Contextual-bandit policy for proactive suggestions
- Safety critic for irreversible or privacy-sensitive actions

Evaluation and benchmarking: Task success and intervention rate over 30-day simulated traces, preference regret, deletion compliance, privacy leakage, p95 latency and cost per user-day.

Outcome: An end-to-end personal AI operating system with measured personalization, bounded autonomy and auditable privacy controls.

18. AI-Native Compliance Infrastructure for Multi-Jurisdiction Finance [YC]

Build compliance infrastructure that monitors regulatory change across jurisdictions, maps obligations to a company's actual operations, flags anomalies and produces audit-ready evidence.

Why it matters: YC Fall 2026 RFS calls this out: compliance is still spreadsheets and headcount, and complexity compounds with every new market entered.

Technologies: LLMs for regulatory text, retrieval over legal corpora, obligation extraction and normalization, anomaly detection over transactions, provenance and audit logging, policy-as-code.

Tasks:

- Ingest and version regulatory texts across several jurisdictions
- Extract structured obligations and detect changes between versions
- Map obligations onto a simulated company's controls and transactions
- Train four models for change detection, obligation classification and anomaly flagging
- Generate audit-ready reports with full provenance for every assertion

Models to build and compare (4 minimum):

- Obligation-extraction model over regulatory text
- Regulatory-change detection and diff-classification model
- Transaction anomaly detector
- Retrieval-augmented compliance question-answering model

Evaluation and benchmarking: Obligation-extraction F1 against annotated regulation, change-detection recall, anomaly precision at fixed alert budget, expert review of generated reports.

Outcome: A compliance platform that reduces manual review time with measured accuracy and complete audit trails.

19. Multimodal Accessibility and Navigation Companion [YC]

Build a phone-and-wearable assistant that helps blind or low-vision users understand dynamic scenes, navigate indoor spaces and complete tasks through low-latency spoken guidance.

Why it matters: YC Fall 2026 highlights AI products that change how people get around and stay independent. Accessibility makes reliability, latency and personalization non-negotiable.

Technologies: Vision-language models, egocentric video, visual SLAM, temporal scene graphs, speech, active perception, uncertainty calibration and on-device distillation.

Tasks:

- Assemble a consented or synthetic benchmark of routes, hazards and tasks
- Ground spoken descriptions in tracked objects and free-space geometry
- Fuse visual SLAM with semantic landmarks for indoor navigation
- Add confidence-aware clarification and safe-abstention behavior
- Compare server, edge and hybrid variants under a strict latency budget

Models to build and compare (4 minimum):

- Vision-language grounded scene describer
- Temporal scene-graph tracker
- Risk-aware semantic route planner
- Calibrated clarification and abstention policy

Evaluation and benchmarking: Hazard recall, route success, localization error, phrase grounding, calibration, response latency and battery use; user study only with approval.

Outcome: A real-time accessibility companion with reproducible safety tests, explicit uncertainty and deployable edge performance.

20. Privacy-Preserving Stablecoin Settlement and Proofs [YC]

Build a stablecoin settlement protocol that proves authorization, policy compliance and correct routing without revealing transaction amounts, counterparties or agent strategies.

Why it matters: YC Fall 2026 highlights stablecoins and agentic commerce. Institutional adoption needs programmable controls and audit evidence without exposing sensitive payment graphs.

Technologies: Zero-knowledge proofs, verifiable credentials, threshold signatures, confidential transactions, policy circuits, testnets, formal verification and adversarial simulation.

Tasks:

- Define a payment-policy language for limits, roles, sanctions and approvals
- Compile policies into proofs that reveal only pass/fail and required evidence
- Implement delegated agent spending with threshold authorization and revocation
- Add multi-rail settlement with proof-carrying audit records
- Red-team replay, front-running, key compromise and privacy leakage

Models to build and compare (4 minimum):

- Graph model for private counterparty-risk features
- Sequence anomaly detector over selectively disclosed events
- Learned proof-routing and batching policy
- Rule-based policy circuit and threshold-signature baseline

Evaluation and benchmarking: Proof generation and verification latency, throughput, disclosure leakage, policy coverage, fraud recall, settlement cost and attack resistance.

Outcome: A proof-carrying settlement layer for stablecoins and agents with measurable privacy, compliance and performance.

21. Consumer-Scale Personal Agent with Sub-Cent Unit Economics [YC]

Build a personal assistant good enough to use every day, then drive its cost per user per day down by two orders of magnitude without losing task quality.

Why it matters: YC's Fall 2026 consumer RFS argues the consumer moment arrives when the magic stops costing roughly \$1,000 per user per month. Cost, not capability, is the binding constraint.

Technologies: Model routing and cascades, semantic caching, distillation to small models, on-device fallback, speculative decoding, prompt compression, per-request cost accounting and telemetry.

Tasks:

- Define a daily-use task suite and the quality bar a real user would accept
- Build the assistant end to end, with tool use and persistent memory
- Implement four cost-reduction strategies and measure quality under each
- Track cost per completed task, not cost per token
- Report the quality-cost frontier and the point where quality collapses

Models to build and compare (4 minimum):

- Frontier-model baseline (quality ceiling, cost ceiling)
- Router / cascade that escalates only the hard queries
- Distilled small model serving the common path
- Semantic cache with learned hit prediction

Evaluation and benchmarking: Task success on the suite versus cost per user-day; p95 latency; cache hit rate; the cost level at which success falls below the acceptance bar.

Outcome: A working personal agent with a published cost-quality frontier and a defensible answer to what one user costs per month.

Propose your own topic

Send your proposal by email and get it approved before you start.

A proposal must clear the same bar as the topics in this deck:

- A problem that is genuinely open — not a tutorial reimplementation
- An end-to-end system, with the architecture sketched
- Four or more distinct models you will build and compare
- A named benchmark or dataset, and the metrics you will report
- A realistic compute budget and data-access plan
- Enough work to occupy 4–5 people for two semesters

Proposals that are a wrapper around one API call will not be approved.